
AI Product Production Factory

Detailed Technical Specification — Implementation Contract

Owner	Jon — FascinateCopy
Document	Implementation-level specification (build directly against this)
Version	1.1 — Claude Code runtime; scope Phases 3–4
Build scope	Phases 3–4 (+ brief input contract): asset generation · hierarchy validation · gated approval
Deferred	Phases 5–6 (dashboard, weekly report) — Appendix A; need a live product's data
Stack	Python 3.11 · Claude Code CLI (Max login, no API key) · file-based gates
Upstream / downstream	Grand Slam Offer GPT → this system → Maria Wendt Sales Machine

Contents

- 1 Overview & scope · 2 Architecture · 3 Asset hierarchy model
- 4 Data models (every artifact schema)
- 5 Offer Brief contract (Phase 2) · 6 Generation engine (Phase 3)
- 7 Asset generation catalog — per-asset specs
- 8 Hierarchy validator (Phase 4) · 9 Approval gates · 10 Pipeline
- 11 CLI · 12 Output layout · 13 Configuration · 14 Failure taxonomy · 15 Non-functional
- 16 Constraints · 17 Project structure · 18 Setup · 19 Human-only · 20 Roadmap · Appendix A (5–6)

1. Overview & Scope

The Factory takes one input — an approved **Offer Brief** from the Grand Slam Offer GPT — and produces the complete, hierarchy-checked asset package needed to sell and deliver a low-ticket digital product through the Maria Wendt Sales Machine. Target: ~90% AI production, ~10% human review at defined gates. It never creates offers and never publishes; it generates and gates.

This build is Phases 1-4: (1) methodology grounding, (2) the brief contract + validator, (3) the asset generation engine, (4) the hierarchy validator — brief in, full asset package out, gated for approval. **Phases 5-6** (live-campaign dashboard + weekly report) are deferred to Appendix A: they measure a running funnel and have no data to work on until a product is live.

2. Architecture

A Python engine is the source of truth: it owns the brief contract, the generation prompts, the validator, the gate store, and output rendering. Approval is file-based (a JSON gate store), which is auditable and later swappable for a Google Sheet cell without changing pipeline code. Claude is Generation runs through the Claude Code CLI (claude -p) on the machine's logged-in Max account — no API key and no per-call billing.

Component	Responsibility
config.py	Typed, .env-driven settings; fails loud on a missing key.
factory/brief.py	Phase 2: OfferBrief schema + completeness validation + rejection.
factory/llm.py	Claude client wrapper; JSON-mode + repair; normalized LLMError.
factory/prompts/*	Phase 1 grounding + per-asset prompt builders.
factory/generators.py	Phase 3: generate-and-normalize functions per asset.
factory/hierarchy_validator.py	Phase 4: structural + AI checks → Findings.
factory/gates.py	Gate store + state machine + GateBlocked.
factory/output.py	Render each artifact to json (truth) + md (review).
factory/pipeline.py	Ordered, resumable, gated orchestration.
run.py	CLI surface.

3. Asset Hierarchy Model & Invariants

- **Core Product ▶ Core Deliverables ▶ Bonus Deliverables ▶ Order Bump ▶ Future Upsells** (planned only) ▶ Content ▶ DM ▶ Checkout ▶ Delivery assets.
- Bonuses never outshine the core. The bump never replaces the core. Upsells are planned, not built, until validation. Every deliverable serves the promise. No duplicates.

These invariants are enforced twice: as generation-time guardrails in every prompt, and as post-generation checks in the hierarchy validator (§8).

4. Data Models

Every generated artifact conforms to one of these schemas. Generators request strict JSON in these shapes; the output layer renders each to <name>.json (machine truth) and <name>.md (human review).

OfferBrief (input)

Field	Type	Notes
offer_name	string	Required. Slugs the output folder.
avatar	string	Required. Who the product is for.
main_problem	string	Required.
main_promise	string	Required. Gated at product_promise.
core_product_type	string	Required. e.g. mini-course, template pack.
core_deliverables	list[str obj]	Required, ≥ 1 item.
bonus_deliverables	list[str obj]	May be empty.
order_bump	string object	Required; use 'none' if absent.
future_upsells	list	May be empty. Planned only.
buyer_objections	list[str]	Required, ≥ 1 item.
roi_logic	string	Required.
tone	string	Required. Voice for all copy.
ethical_boundaries	string	Required. Hard limits.
delivery_format	string	Required.
extras	object	Optional. Preserved, not validated.

ProductPackage

Field	Type	Notes
product_outline	string	2-4 sentence overview.
modules	list[Module]	Module = {title, outcome, lessons[]}. Lesson = {title, teaching_point, format}.
modules[].lessons	list[Lesson]	
worksheets	list[{name,purpose}]	
templates	list[{name,purpose}]	
quick_start_guide	string	First-win path, <30 min.
customer_success_path	list[{milestone, how_they_know_they_hit_it}]	

Field	Type	Notes
delivery_instructions	string	Matched to delivery_format.

CoreDeliverable

Field	Type	Notes
name	string	
purpose	string	Job it does for the buyer.
format	string	Concrete artifact type.
problem_solved	string	
completion_criteria	string	How buyer knows they're done.
delivery_asset	string	Artifact + how delivered.

BonusDeliverable

Field	Type	Notes
name	string	
purpose	string	Acceleration or objection removal.
objection_removed	string	Ties to buyer_objections.
why_it_belongs_as_a_bonus	string	Why not core / not standalone.
delivery_asset	string	

OrderBump

Field	Type	Notes
product	string	'none' if no bump.
copy	string	On-checkout copy, in Tone, ethical.
positioning	string	Why it's the obvious add-on; stays subordinate.
delivery_method	string	

ReelIdea

Field	Type	Notes
index	int	1-based, unique across batches.
angle	string	Psychological angle, 2-4 words.
hook	string	Scroll-stopping opener.
core_idea	string	What the reel shows/says.

Field	Type	Notes
cta_keyword	string	Single DM keyword it drives to.

ReelScript

Field	Type	Notes
index	int	Matches source ReelIdea.
hook	string	Spoken + on-screen opener (0-3s).
script	string	Full voiceover, line by line.
b_roll_or_visual_notes	string	On-screen per part.
on_screen_text	list[str]	Caption overlays in sequence.
spoken_cta	string	Closing line to the keyword.
cta_keyword	string	

CarouselOutline

Field	Type	Notes
index	int	
angle	string	
hook_slide	string	Slide 1 headline.
slides	list[{headline, body}]	Body slides.
cta_slide	string	Final slide CTA.
cta_keyword	string	

SocialPost (pitch / nurture / objection / proof)

Field	Type	Notes
index	int	
post_type	enum	direct_pitch nurture objection proof.
hook	string	First line.
body	string	Caption body.
cta	string null	null for nurture.
cta_keyword	string null	
objection_addressed	string?	objection posts only.
proof_type	enum?	proof only: testimonial result process screenshot_prompt.
proof_content	string?	proof only; screenshot_prompt = what to capture.

StoryPrompt

Field	Type	Notes
index	int	
frame_type	enum	poll question quiz real_countdown dm_trigger.
prompt_text	string	
sticker	string	Sticker/interaction to use.
cta_keyword	string null	

CTACaption / DMKeywordPrompt / ManyChatCopy

Field	Type	Notes
cta_captions	list[{index, for_asset, caption, keyword}]	for_asset: reel carousel post.
dm_keyword_prompts	list[{keyword, triggered_by, intent, first_response_ref}]	
manychat	{flow_name, trigger_keyword, steps[], handoff_condition}	step = {order, message, quick_replies[], delay}.

DMSequence

Field	Type	Notes
keyword_list	list[str]	All keywords used across content.
initial_dm	string	Delivers promised value + qualifies.
follow_ups	list[{order, message, send_after}]	
checkout_messages	list[str]	
reminder_messages	list[{order, message, send_after}]	
objection_responses	list[{objection, response}]	Maps buyer_objections.

CheckoutAssets

Field	Type	Notes
checkout_page_copy	string	

Field	Type	Notes
product_description	string	
bullet_stack	list[str]	Benefit bullets.
bonus_copy	string	Checkout bonus stack copy.
order_bump_copy	string	
thank_you_page	string	
delivery_email	{subject, body}	
purchase_confirmation	{subject, body}	
follow_up_emails	list[{order, subject, body, send_after}]	
refund_instructions	string	Respects ethical_boundaries.

Finding (validator output)

Field	Type	Notes
check	string	Check identifier.
severity	enum	blocking warning info.
message	string	What's wrong.
recommended_fix	string	Actionable remedy.

5. Offer Brief Contract (Phase 2)

Input is one JSON object matching OfferBrief (§4). Validation rules:

- String fields must be present and non-blank.
- core_deliverables and buyer_objections must be lists with ≥ 1 item.
- bonus_deliverables and future_upsells may be empty lists.
- order_bump is required; the literal 'none' is valid.
- Unknown fields are preserved under extras and never block validation.

Rejection behaviour: if any rule fails, the Factory raises IncompleteBriefError and returns a numbered list of exactly what's missing, to be sent back to the Grand Slam Offer GPT. No generation runs on an incomplete brief.

Rejection output (example)

```
Offer Brief is incomplete. Send it back to the Grand Slam Offer GPT with these missing items:  
- Missing: Main Promise.  
- Missing or empty: Core Deliverables (needs at least one item).  
- Missing: ROI Logic.
```

Minimal valid brief (example)

```
{  
  "offer_name": "The 20-Minute Reels System",  
  "avatar": "Time-poor coaches with <5k followers",  
  "main_problem": "They post but nothing converts to DMs or sales",  
  "main_promise": "Book 5 sales conversations a week from Reels",  
  "core_product_type": "mini-course + templates",  
  "core_deliverables": ["Reel hook framework", "Keyword-to-DM setup"],  
  "bonus_deliverables": ["30 done-for-you hooks"],  
  "order_bump": "Caption pack ($9)",  
  "future_upsells": ["Group coaching"],  
  "buyer_objections": ["No time", "Tried Reels before, failed"],  
  "roi_logic": "One client pays back the price 50x",  
  "tone": "Warm, direct, no hype",  
  "ethical_boundaries": "No income guarantees",  
  "delivery_format": "Google Drive folder + PDF"  
}
```

6. Generation Engine (Phase 3)

Prompt architecture

Every generator builds a (system, user) pair. The **system** prompt is assembled once per role: role line + methodology grounding + hierarchy rules + ethics rules + (optional) injected course notes. The **user** prompt carries the approved brief as JSON, the task, the exact output schema, and a strict 'JSON only — no fences, no commentary' instruction.

```
system = role + METHODOLOGY_BASE + HIERARCHY_RULES + ETHICS_RULES + [course_notes]
user   = brief_context(brief) + task + output_schema + "Respond with ONLY valid JSON"
```

LLM call contract

- Generation shells out to ``claude -p --output-format json`` on the Max login; the `.result`` field is parsed. No API key. `--bare` is deliberately NOT used (it would force a key).
- `generate()` returns text; `generate_json()` strips ``` fences and parses; on failure it extracts the outermost JSON object/array; if that fails it raises `LLMError`.
- All provider/network errors are normalized to `LLMError` at the client boundary.

Batching & volume

High-volume content is generated in batches to protect quality and avoid truncation. Reel ideas use a `start_index` offset so batches don't repeat; scripts are generated per idea-batch. Normalizers tolerate `{"items": [...]}` wrappers and drop non-dict entries.

Idempotency & resume

Each stage writes `<stage>.json`. On re-run, an existing file is loaded instead of regenerated (no wasted tokens) unless `--force` is passed. This is what makes the gated pipeline cheap to resume after each approval.

7. Asset Generation Catalog (per-asset specs)

Each block is the contract for one generator: what it produces, how much, batching, output schema (see §4), the key constraints its prompt enforces, and its role in the funnel.

Reel Ideas

Purpose	Top-of-funnel hooks that pull the avatar toward the promise and set up a keyword.
Quantity	30
Batching	3 batches of 10; start_index offsets the prompt so batches don't repeat angles.
Output schema	ReelIdea[]
Key constraints	Vary angle across the set (pain, myth-bust, quick win, before/after, mistake, contrarian, day-in-life, results). Each maps to exactly one DM keyword.
Funnel role	Attention → keyword capture.

Reel Scripts

Purpose	Shootable script for each idea.
Quantity	30
Batching	Scripts generated per idea-batch (10 at a time) from the ideas array.
Output schema	ReelScript[]
Key constraints	20-45s spoken. Hook in first 3s. Ethical CTA to comment/DM keyword. No unsupported claims.
Funnel role	Attention → keyword capture.

Carousel Outlines

Purpose	Authority-building teaching carousels.
Quantity	15
Batching	2 batches (~8 + 7).
Output schema	CarouselOutline[]
Key constraints	One idea taught deeply per carousel; 6-10 slides; final slide CTA to keyword.
Funnel role	Trust → keyword capture.

Direct Pitch Posts

Purpose	Posts that name the offer and outcome plainly.
Quantity	15
Batching	2 batches.

Output schema	SocialPost[] (post_type=direct_pitch)
Key constraints	Plain-spoken offer + outcome + CTA. No hype, no fake scarcity.
Funnel role	Consideration → CTA.

Nurture Posts

Purpose	Relate and build trust; no ask.
Quantity	15
Batching	2 batches.
Output schema	SocialPost[] (post_type=nurture)
Key constraints	cta and cta_keyword are null. Story/relatability, ties to the avatar's world.
Funnel role	Trust.

Objection Posts

Purpose	Dissolve one specific hesitation each.
Quantity	10
Batching	1 batch.
Output schema	SocialPost[] (post_type=objection)
Key constraints	Each post maps to one item in buyer_objections via objection_addressed.
Funnel role	De-risk → CTA.

Proof Posts

Purpose	Show substantiable evidence.
Quantity	10
Batching	1 batch.
Output schema	SocialPost[] (post_type=proof)
Key constraints	proof_type ∈ {testimonial, result, process, screenshot_prompt}. Only claims that can be substantiated; screenshot_prompt states what real asset to capture — never fabricated.
Funnel role	Credibility → CTA.

Story Prompts

Purpose	Quick engagement + keyword triggers for Stories.
Quantity	10
Batching	1 batch.
Output schema	StoryPrompt[]

Key constraints	frame_type drives the interaction; real_countdown only for genuine deadlines.
Funnel role	Engagement.

CTA Captions

Purpose	Reusable caption CTAs per asset type.
Quantity	set
Batching	1 call.
Output schema	cta_captions[]
Key constraints	Map to reel/carousel/post; each names its keyword.
Funnel role	Conversion nudge.

DM Keyword Prompts

Purpose	The keyword→intent map.
Quantity	set
Batching	1 call.
Output schema	dm_keyword_prompts[]
Key constraints	Each keyword ties to the content that triggers it and the DM entry point.
Funnel role	Capture routing.

ManyChat Copy

Purpose	Automation flow copy for the DM tool.
Quantity	set
Batching	1 call.
Output schema	manychat{}
Key constraints	Steps with quick replies + delays; explicit handoff_condition to a human for nuance.
Funnel role	Automated selling.

Product Package

Purpose	The core product itself.
Quantity	1
Batching	1 call.
Output schema	ProductPackage
Key constraints	Fully delivers main_promise; completable at a low-ticket price; no filler modules.
Funnel role	The product.

Core Deliverables

Purpose	Each brief core deliverable, fully specified.
Quantity	= brief count
Batching	1 call.
Output schema	CoreDeliverable[]
Key constraints	Does not invent or drop deliverables; each has completion_criteria.
Funnel role	Delivery.

Bonus Deliverables

Purpose	Each brief bonus, fully specified.
Quantity	= brief count
Batching	1 call.
Output schema	BonusDeliverable[]
Key constraints	Never more valuable/desirable than core; each removes a named objection.
Funnel role	Objection removal.

Order Bump

Purpose	The checkout add-on.
Quantity	1
Batching	1 call.
Output schema	OrderBump
Key constraints	Complements, never replaces the core; small fraction of core price.
Funnel role	AOV lift.

DM Automation Sequence

Purpose	The conversation that sells.
Quantity	1
Batching	1 call.
Output schema	DMSequence
Key constraints	Value-first, non-spammy; objection_responses map buyer_objections; human handoff implied.
Funnel role	Selling. Gated: dm_automation.

Checkout Assets

Purpose	All checkout + delivery copy (10 sub-assets).
Quantity	1 bundle
Batching	1-2 calls (may split page-copy from emails).
Output schema	CheckoutAssets
Key constraints	Accurate to the product; ethical refund terms; delivery matches delivery_format.
Funnel role	Conversion + delivery. Gated: checkout_copy.

8. Hierarchy Validator (Phase 4)

Runs after generation. Structural checks are deterministic Python; judgment checks call Claude with a validator system prompt and the generated assets, returning Finding[] (see §4). Aggregation: result.ok == (no blocking findings). The hierarchy_validation gate cannot be approved while any blocking finding stands — the pipeline writes the fix list and stops; fix, then re-run with --force.

Check	Method	Trigger	Severity	Recommended fix
duplicate_deliverables	Structural (difflib ratio ≥ 0.85 on names across core+bonus)	Two deliverables read as near-identical	warning	Merge, or differentiate the problem each solves.
missing_deliverables	Structural	core_deliverables empty OR product_package.modules empty (blocking); no quick_start (warning)	blocking / warning	Regenerate the empty section (use --force).
bonus_stronger_than_core	AI judge (product_package + core + bonuses)	A bonus is more valuable/desirable than the core	warning→blocking	Rebalance or demote the bonus; strengthen the core.
order_bump_too_valuable	AI judge (order_bump + core)	The bump rivals or could replace the core	warning→blocking	Shrink the bump's scope.
generic_ai_content	AI judge (sample of content assets)	Copy is templated / not specific to this avatar+promise	warning	Regenerate specified to the avatar and promise.
unsupported_claims	AI judge (all copy; special focus health + income/ROI)	Misleading, exaggerated, unsupported, or fabricated income/health claims	blocking	Remove or substantiate; caveat honestly.
support_burden	AI judge (deliverables)	A deliverable likely to spike confusion / support tickets	warning	Simplify the deliverable.
promise_misalignment	AI judge (deliverables vs main_promise)	A deliverable doesn't serve the Main Promise	warning	Retie to the promise or cut it.

AI judge output contract

```
[ { "check": "bonus_stronger_than_core", "severity": "warning",  
  "message": "...", "recommended_fix": "..." } ] // [] = all clear
```


9. Approval Gates

A gate is a named checkpoint. State lives in `_gates/gates.json` as `gate → pending | approved | rejected`. The pipeline writes a `REVIEW__<gate>.md` artifact at each gate and refuses to pass until the gate is approved (raises `GateBlocked`). Approving is one CLI command; the same interface can later read a Google Sheet cell instead of a file.

#	Gate	Guards	Approve after reviewing...
1	offer_brief	Right, complete input	offer_brief.json
2	product_promise	The transformation	REVIEW__product_promise.md
3	core_deliverables	Core complete & on-promise	core_deliverables.md
4	bonus_deliverables	Bonuses stay subordinate	bonus_deliverables.md
5	order_bump	Bump subordinate & logical	order_bump.md
6	dm_automation	On-brand, non-spammy DMs	dm_automation.md
7	checkout_copy	Accurate, ethical checkout	checkout.md
8	hierarchy_validation	No blocking issues	hierarchy_validation.md
9	health_claims	No unsupported health claims	hierarchy_validation.md
10	roi_claims	No unsupported income/ROI claims	hierarchy_validation.md
11	final_product	Whole package production-ready	final summary

Gate set is fixed; the live ordering is defined by the pipeline sequence in §10.

10. Pipeline Sequence

The orchestrator advances until the first pending gate, writes that gate's review file, and stops with instructions. Approve, re-run, and it resumes from cache. `--force` regenerates a stage.

#	Stage	Produces	Gate before continuing
1	Brief load + completeness	validated brief (Phase 2)	— hard reject if incomplete
2	Brief confirm	offer_brief.json	offer_brief
3	Promise confirm	— (surfaces promise)	product_promise
4	Product package	product_package.json/.md	—
5	Core deliverables	core_deliverables.json/.md	core_deliverables
6	Bonus deliverables	bonus_deliverables.json/.md	bonus_deliverables
7	Order bump	order_bump.json/.md	order_bump

#	Stage	Produces	Gate before continuing
8	Content pack	reels · carousels · posts · stories · ctas · keywords · manychat	—
9	DM automation	dm_automation.json/.md	dm_automation
10	Checkout assets	checkout.json/.md	checkout_copy
11	Hierarchy validation	hierarchy_validation.json/.md (Phase 4)	hierarchy_validation · health_claims · roi_claims
12	Final	final summary	final_product

11. CLI Reference

Command	Does	Output	Exit codes
validate --brief PATH	Phase 2 check only; no API calls.	'Brief is complete' or numbered missing list.	0 ok · 1 incomplete
generate --brief PATH [--force]	Run the pipeline to the next pending gate.	Writes assets; stops at first pending gate with review path.	0 done · 3 paused at gate · 1 incomplete · 4 blocking findings
approve --offer SLUG --gate GATE	Approve one gate.	Updates gates.json.	0 · 2 bad gate
reject --offer SLUG --gate GATE	Reject one gate (blocks progression).	Updates gates.json.	0
status --offer SLUG	Show gate states.	Gate table (pending/approved/rejected).	0
regenerate --brief PATH --stage NAME	Force-regenerate one stage.	Rewrites that stage's files.	0

12. Output Layout (per run)

```
output/<offer-slug>/
  offer_brief.json
  product_package.json | .md      core_deliverables.json | .md
  bonus_deliverables.json | .md   order_bump.json | .md
  reels.json | .md   carousels.json | .md   posts.json | .md
  story_prompts.json | .md   cta_captions.json | .md
  dm_keyword_prompts.json | .md   manychat.json | .md
  dm_automation.json | .md   checkout.json | .md
  hierarchy_validation.json | .md
  _gates/
    gates.json
    REVIEW__<gate>.md ...
```

13. Configuration Reference (.env)

Variable	Default	Meaning
CLAUDE_MODEL	(blank)	Optional. Pin a model, else use the Claude Code session default.
CLAUDE_TIMEOUT_SECONDS	300	Max wait per generation call.
LLM_MAX_TOKENS	8000	Per-call output ceiling.
LLM_TEMPERATURE	0.7	Lower for more deterministic runs.
OUTPUT_DIR	output	Per-offer asset folders.
BRIEFS_DIR	briefs	Input briefs.

Variable	Default	Meaning
METHODOLOGY_NOTES_PATH	(blank)	Optional file of your extracted course notes; injected as grounding.

14. Failure Taxonomy

Failure	Type	Behaviour
Incomplete brief	IncompleteBriefError	Refuse to run; print the missing-field list. Exit 1.
LLM / network error	LLMError	Normalized + raised; already-generated stages stay cached on disk.
Bad JSON from model	LLMError (after repair)	Strip fences, extract outermost JSON; if still bad, error loudly.
Blocking validation	GateBlocked (hierarchy)	Stop before the gate; write fix list; re-run --force after edits.
Gate pending	GateBlocked	Pause (not an error). Print review path + approve command. Exit 3.

15. Non-Functional Notes

- **Cost/volume:** a full product run is ~25–31 Claude Code calls. Covered by the Max subscription (no per-call API billing). Max supports 1–3 steady headless agents; heavier parallelism would need an API key instead.
- **Rate limits & retries:** calls run sequentially; add exponential-backoff retry on 429/5xx (to build). Cached stages mean a mid-run failure never loses completed work.
- **Determinism:** temperature 0.7 is non-deterministic; lower it for reproducibility. Caching gives stable outputs across resume runs.

16. Constraints & Ethics

- Never invent offers; work strictly from the approved brief.
- Never replace the Sales Machine — only accelerate production feeding into it.
- No fake urgency/scarcity; deadlines and limits must be real.
- No misleading/exaggerated/unsupported claims; no fabricated income or health claims.
- No spammy automation; DMs deliver value and respect the recipient.
- Products must be genuinely valuable and production-ready.
- The brief's ethical_boundaries override defaults where stricter.

17. Project Structure

```
product-factory/
├── run.py                CLI
├── config.py             .env-driven settings (Claude only)
├── requirements.txt      .env.example
├── briefs/example_offer_brief.json
├── docs/build_spec.py    regenerates this PDF
├── factory/
│   ├── brief.py          Phase 2
│   ├── llm.py            Claude client
│   ├── generators.py     Phase 3
│   ├── hierarchy_validator.py Phase 4
│   ├── gates.py          output.py pipeline.py
│   └── prompts/ (methodology, product_package, content_pack, dm_automation, checkout)
└── output/              generated assets per offer
```

18. Setup & Run (Windows / PowerShell)

```
py -3.11 -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
npm install -g @anthropic-ai/claude-code # then run `claude` once to log in
Copy-Item .env.example .env             # no API key needed
python run.py doctor                     # verify Claude Code is installed + logged in
python run.py validate --brief .\briefs\example_offer_brief.json
python run.py generate --brief .\briefs\example_offer_brief.json
python run.py approve --offer <slug> --gate offer_brief
python run.py generate --brief .\briefs\example_offer_brief.json # resumes
```

Verify at each step: (.venv) in the prompt; 'Brief is complete'; output\\ fills with json+md; the run stops at the first gate with a review path.

19. What Only You Can Do

- Paste your Anthropic API key into .env (secrets never enter source or the tool).

- Provide a real approved Offer Brief from the Grand Slam Offer GPT (example ships for testing).
- Optionally paste extracted Maria Wendt course notes to ground generation.
- Authenticate delivery/marketing tools (Instagram, ManyChat, checkout, email, Google).
- Click the approval gates — the 10% human review.

20. Build Roadmap (Phases 1-4)

Stage	Scope	Status
Engine spine	config · brief · llm · pipeline · gates · output	Done
Generation v1	product · core · bonus · bump · reels	Done
Generation v2	carousels · post types · stories · CTAs · keywords · ManyChat	Next
DM + checkout	DM sequence · all checkout/delivery assets	Next
Validator wiring	AI checks on the full asset set	Next
Handoff	HANDOFF.md — 1-hour fresh-machine setup	Last

Deferred (needs a live product): Phase 5 dashboard, Phase 6 weekly report — see Appendix A.

Appendix A — Deferred: Phases 5-6

Included for the full-project picture. Not built in this cycle — both measure a live campaign and have no data until a product is published and selling.

Phase 5 — Tracking Dashboard (Google Sheet from an xlsx builder)

- Product · avatar · price · core deliverables · bonuses · order bump
- Content published · reel views · carousel engagement
- DMs · keyword triggers · checkout clicks
- Purchases · order-bump purchases · revenue · refund rate
- Support issues · customer feedback

Phase 6 — Weekly Optimization Report (from dashboard data)

- Best-performing content · highest-converting DMs · common objections
- Checkout improvements · bonus effectiveness · core product performance
- Kill / Improve / Scale recommendations
- Upsell-unlock recommendation (are validation criteria met yet?)

Trigger: first product live → build Phase 5 → accumulate data → build Phase 6.